# Proposal: slice `evaluator.fs`, and what to do with `scheduler.fs`

Status: **Part 1 implemented** (`evaluator.fs` 850 → 186 lines across two commits). **Part 2 implemented as Option A**, then **superseded by W5**: `scheduler.fs` was rewritten as the live per-process scheduler (state threaded through `EvaluationState`, no module-global mutable), the `spawn`/`yield`/`halt`/ `actors`/`resume`/`kill` words are wired in, and budget exhaustion yields instead of failing. The ratchet now pins only `fork`/`channel`/`send`/`receive` as unimplemented (W6) and tests the scheduler's real behaviour, including a two-process interleaving.

## Part 1 — The five concerns in `evaluator.fs`

Read top to bottom, the file is already *organised* by concern — it has banner comments for each. It is not tangled; it is **unseparated**. The banners are:

| # | Concern | Lines (approximate) | What it holds |
|---|---|---|---|
| 1 | **Program counter and word dispatch** | 799–850 | `run` (the loop), `runOrFail`, `initial`, `EvaluationException` |
| 2 | **Stack shuffling** | 255–292 | `swapWord`, `overWord`, `rotateWord` |
| 3 | **Control: quotation, if, apply, define** | 160–174, 293–358 | `collectQuotation`, `executeIf`, `executeApply`, `executeDefine` |
| 4 | **Bookmarks (proto event-stream)** | 359–429 | `createBookmark`, `executeJump`, `copyBookmark`, `showBookmark` |
| 5 | **Environments: bindings + unification** | 430–510 | `bindWord`, `lookupWord`, `unifyWord`, `resolveWord`, `termWord`, `typedWord`, `variableNameOfText` |
| 6 | **Vivid: models and set algebra** | 511–666 | `popModel`, `modelWord`, `admitWord`, `andWord`, `orWord`, `norWord`, `isWord` |
| 7 | **Shared plumbing** | 11–158, 175–254 | `locate`, `findPair`, `addToEnvironment`, `writeSet`, `isMemberId`, `popResolved`, `binaryOperation`, `valueOf` |

That is seven blocks, but **five concerns**: 1 is the driver, 7 is plumbing, and 2–6 are five feature families. The plumbing (7) is the reason the features cannot be split *today*: `writeSet`/`isMemberId`/ `findModel` are Vivid's, but `addToEnvironment`/`findPair` are environments', and both sit in one block.

### Proposed slices

**Implemented.** The final layout, in `bases/scale-cli/scale.fsproj` order, all under `components/prelude/evaluator/`:

```
components/prelude/evaluator/common.fs       (135)  shared plumbing: Railway helpers,
association vectors
components/prelude/evaluator/stack.fs         (60)  swap, over, rotate, duplicate,
drop
components/prelude/evaluator/control.fs      (105)  quotation collection, if, apply,
define
components/prelude/evaluator/bookmark.fs      (90)  bookmark, jump, copyBookmark,
showBookmark
components/prelude/evaluator/environment.fs  (428)  bind, lookup, unify, resolve,
term, typed
                                  (and the `?name` sigil/term encoding)
components/prelude/evaluator/vivid.fs        (242)  model, admit, and, or, nor, is
```

```
components/prelude/evaluator/evaluator.fs    (186)  the dispatcher, the main loop,
and the entry points
```

`evaluator.fs` went **850 → 186 lines** across two commits. Every handler is the same code, moved; no body changed.

> **Later slices (added after this snapshot).** The list above is the Part-1 layout. Two more feature slices have since landed: `components/prelude/evaluator/value-sets.fs` (`[ ]` literals, `seal`, `unname`, and the structural set words; W3/D14) and `components/prelude/evaluator/dispatch.fs` (`generic`, `method`, and type-first selection; W10/D19). `executeWord` now calls into eight slices plus `common`; the current file sizes are `sort.fs` 482 and `dispatch.fs` 634 lines, with `evaluator.fs` at 218. The order in `bases/scale-cli/scale.fsproj` is the dependency order.

### The original plan

A vertical slice here means: a module that owns its words, its helpers, and its tests, and exposes only the word handlers. The shared plumbing is the seam.

Each slice is a module of `EvaluationState -> Result<EvaluationState, EvaluationError>` handlers — the exact shape the private `*Word` functions already have — plus the helpers only it uses. `executeWord` becomes a `match` that calls into them, and stays in `evaluator.fs` as the one dispatcher.

### Why this is safe, and what it costs

**Safe**, because:

- Every slice is the *same code*, moved. No handler changes its body.
- The seam is already `EvaluationState -> Result<…>`; nothing new is invented.
- The 197-test suite covers each family (Stack shuffling, Bookmarks, Unification, Vivid, Railway, Reduction budget), so a move that breaks a behaviour fails a named test.
- F# compilation order is explicit in `bases/scale-cli/scale.fsproj`, so the dependency direction is visible and the compiler enforces it.

**Costs**, honestly:

- **Six new files** in `bases/scale-cli/scale.fsproj`, in dependency order. Plumbing (`popResolved`, `valueOf`, `binaryOperation`, `locate`) must be in a module every slice can open — `components/prelude/evaluator/common.fs` before the slices.
- **Some helpers move twice.** `writeSet` is used by Vivid only, so it goes to `components/prelude/evaluator/vivid.fs`; `addToEnvironment` by environments only, to `components/prelude/evaluator/environment.fs`. But `findPair` is used by *both* `bind`/`lookup` and `unify` — so it must be shared, not duplicated. That is the one real design decision.
- **`unifyWord` reads the variable sigil `?x`** (`extractVariableName`), which is a *lexical* concern the tokenizer also touches. Keeping it in `components/prelude/evaluator/environment.fs` is right (it is unification's), but it is worth naming as a cross-cutting read.
- **Diff size**: roughly a 850-line file becomes six files of 60–200 lines plus a ~190-line driver. A large diff, but mechanical.

### My recommendation

Do it, in **two steps**, so a red step is easy to localise:

1. **Extract the two largest, most independent slices first: Vivid and Bookmarks.** Together they are ~250 lines and their only shared dependency is `popResolved`/`findModel`. Prove the suite green, commit.

2. **Then environments, control, and stack.** These are smaller and share more plumbing; doing them after step 1 means the shared module already exists.

Do **not** do all five in one commit: a mechanical move that fails one test is much easier to bisect in two commits than in one.

**What I would NOT do**

- **Not rename the words.** The surface is the surface; this is a file reorganisation, not a language change.
- **Not introduce a typeclass or a word-registry.** `match` on the token is honest and fast; a registry would be a new abstraction to justify, and the ledger already says multiple dispatch (W10) is the real answer for *sort-based* selection. This is just moving code.
- **Not merge slices into one `eval-words.fs`.** That would be the same file with a new name.

## Part 2 — `scheduler.fs`: evaluate, do not delete yet

You asked to keep `scheduler.fs` in place while evaluating it. Here is the evaluation, from the code and from a live demonstration.

**What it is**

32 lines. A module-global `readyQueue: EvaluationState list` and `currentProcess: EvaluationState option`, with `spawn`, `yieldControl`, and `readyCount`. `yieldControl` swaps in the next queued process and resets its reduction budget; with an empty queue it continues the current one.

**The three facts that decide it**

**1. Nothing references it.** `grep -rn 'Scheduler' *.fs` returns exactly one line: the module's own declaration. `evaluator.fs`, `development/scale/tests.fs`, `properties.fs`, `bases/scale-cli/entry.fs`, `demo.fs` — none call it. It is compiled and never invoked.

**2. No program can reach it.** The surface has no word that spawns or yields. Verified in the SageFs REPL against the real evaluator:

```
spawn      -> NOT a word
yield      -> NOT a word
fork       -> NOT a word
process    -> NOT a word
channel    -> NOT a word
send       -> NOT a word
receive    -> NOT a word
```

So a program cannot create a second process, and nothing else calls `spawn`. The queue is unreachable by construction.

**3. Its module-global state leaks across evaluations.** Demonstrated in the REPL — two unrelated interpreter states share the one queue:

```
after A spawns: readyCount = 1
after B spawns: readyCount = 2  (B sees A's process too)
after one yield: readyCount = 1, Reductions reset to 2000
after second yield (unrelated caller): readyCount = 0
```

The ledger already records this as the reason for replacement: *"Current scheduler is to be replaced, not extended (module-global → per-process state); must not be advertised as working."*

### The verdict

`Scheduler` is **not usable today, and cannot become usable without W5.** It is not a partial implementation of the scheduler; it is the *shape* of one, with the one thing that makes it wrong (module-global mutable state) baked in. That is consistent with the ledger's "to be replaced, not extended".

### What to do with it

Three honest options, in order of preference:

**A. Keep the file, mark it as a spec, and make the compiler enforce that it is unused (recommended).** Leave `scheduler.fs` in place as the documented shape of the coming work, but:

- rewrite its banner to say it is a **non-executing design sketch**, not a component — so no reader thinks `yieldControl` is live;
- note in it that it is module-global *on purpose for the sketch*, and that W5 replaces the state with per-process state;
- **do not add `spawn`/`yield` words** (that would advertise a working scheduler the ledger says must not be advertised).

Cost: zero. Benefit: the file stops being misleading without deleting the intent.

**B. Keep it exactly as is.** Defensible — the ledger is honest about it, and the file is inert. The risk is a future reader seeing a `Scheduler` module with `spawn`/`yieldControl` and assuming it works, which is the specific failure the ledger warns against.

**C. Delete it and re-derive it in W5.** Cleanest tree; loses nothing executable. But it throws away a shape you may want to argue with, and you asked to keep it.

### Why not "just wire it up"

The tempting move — add `spawn` and `yield` words and make `run` call `yieldControl` on budget exhaustion — would be **wrong**, and the ledger says why: the state is module-global, so two evaluations (or two REPL evals, or two tests) interleave through one queue, which the demonstration above shows. Wiring it up would convert a dead module into a live bug. The budget-exhaustion path is where it would bite: `ReductionExhausted` should become a *yield*, but with global state a yield from one program would resume another. That is W5, and it needs per-process state (or, better, threading the scheduler through `EvaluationState` as a value, so there is no global at all).

### Recommendation

**Option A**, plus one line in `docs/roadmap.typ`'s W5 row recording that `scheduler.fs` is the sketch it replaces. Then W5 either promotes the file (with per-process state) or deletes it — a decision for W5, made with the real design in front of it.

### Applied

Option A, implemented:

- `scheduler.fs` carries a banner saying it is a **non-executing design sketch**, not a component, with the demonstration above reproduced in the comment and the explicit warning not to add `spawn`/`yield` words.
- `development/scale/tests.fs` pins the claim with a **ratchet**: for each of `spawn`, `yield`, `fork`, `channel`, `send`, `receive`, `Evaluator.executeWord` must answer `UnknownWord`. The ratchet was proved to fail for the right reason — adding a temporary `| "spawn" -> Ok state` arm to the dispatcher turned it red with *"'spawn' must not be a word: Scheduler is a sketch, not a component"* — and then reverted. A guard that cannot fail is decoration; this one can.

**Later.** This section is historical. W5 replaced the sketch as the component (`components/prelude/evaluator/scheduler.fs`, `scheduler.fs`) and D63 added token deadlines and supervision, so the ratchet below no longer holds for `spawn`, `yield`, `channel`, `send`, and `receive`.